University of Toronto

AER1515 Perception for Robotics: Assignment 1

Student : Yang-Chen Lin

October 24, 2025

Contents

1 Answers

1.1 Question 1: Extending to Multi-Class Classification

1.1.1 Training Loss for Binary Classification

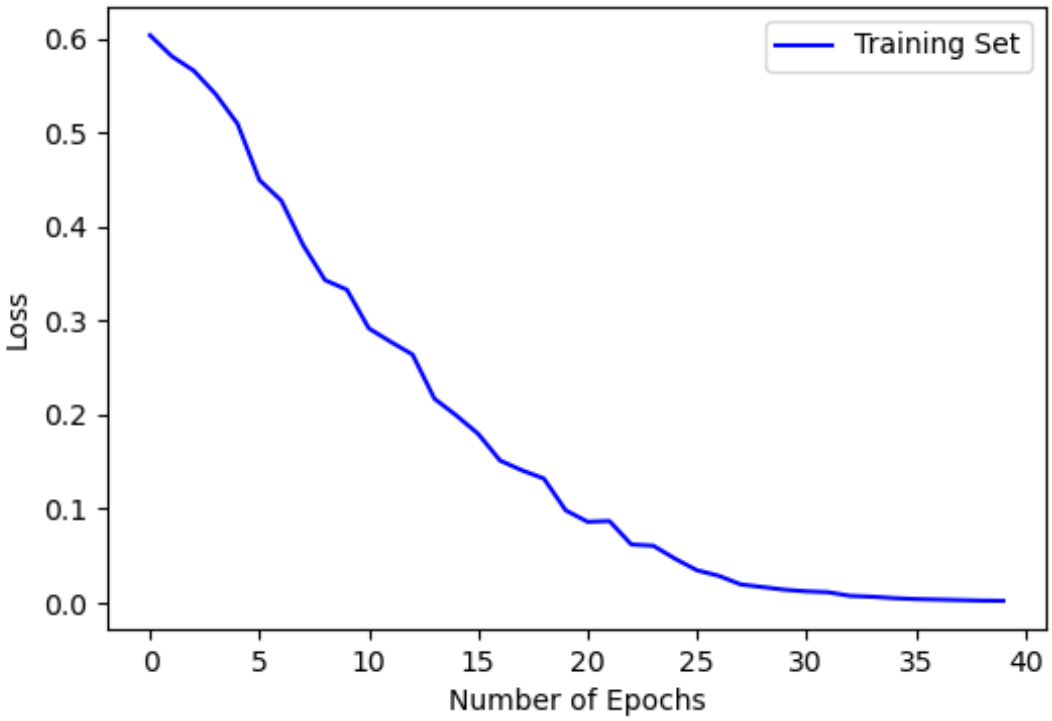

Figure 1: Traning Loss Curve for 40 Epochs

1.1.2 Test Accuracy

The returned accuracy from test.py is at 80% after 40 epochs.

1.1.3 Training Loss and Test Accuracy for Multi-Class Classification

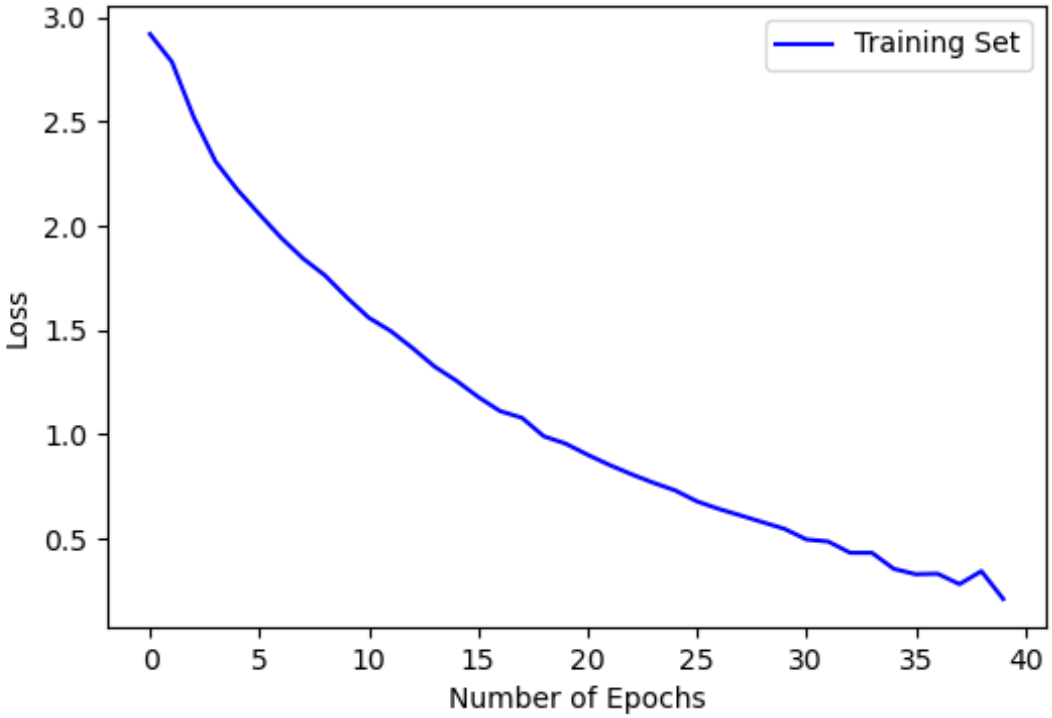

Figure 2: Traning Loss Curve for 40 Epochs

And the test accuracy is at 68% after training for 40 epochs for multi-class classification using the unmodified CNN.

1.2 Question 2: Modifying the CNN Architecture

1.2.1 Training Loss and Test Accuracy after adding Batch-Normalization

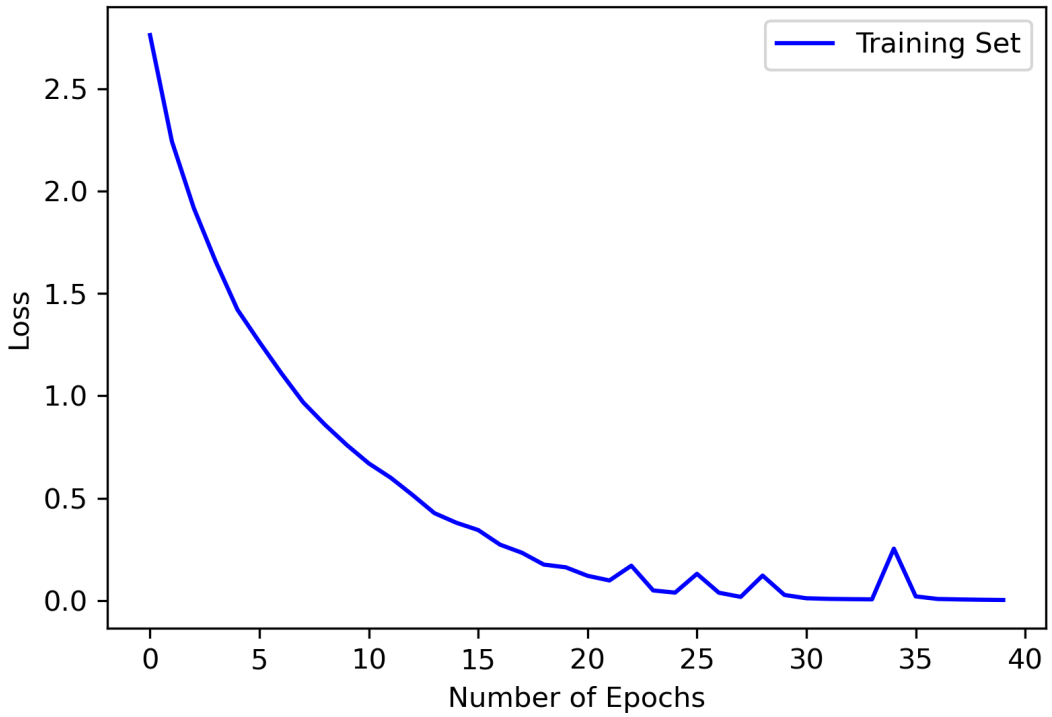

Figure 3: Training Loss Curve of adding a batch-normalization layer after the second convolution layer

The returned test accuracy is 77%, improving from the baseline of 68%. Several configurations of Batch Normalization were tested, and the optimal performance was achieved when a single BN layer was added after the second convolution layer and before the ReLU activation function. Other placements (e.g., after ReLU or in deeper layers) produced inferior results. This improvement suggests that normalizing early feature maps helps stabilize learning and maintain consistent activation scales throughout the network. Moreover, Batch Normalization accelerates convergence (as seen from the training curve) by smoothing the optimization landscape, allowing the network to learn effectively with more stable gradient updates.

1.2.2 Training Loss and Test Accuracy after adding Dropout Layers

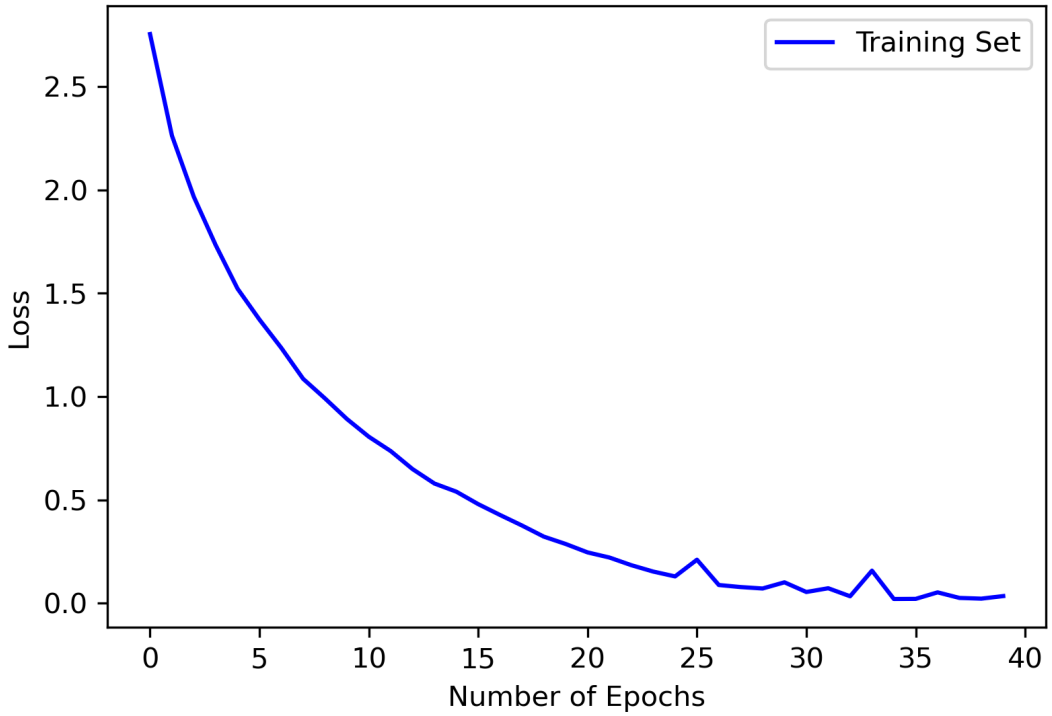

Figure 4: Training Loss Curve of Additional Dropout Layers

After adding dropout after the ReLU activation of the fourth convolution layer and the first fully connected layer, the test accuracy decreased slightly to 75%, which is expected. Dropout randomly disables a subset of neurons during training, forcing the network to explore more representations rather than relying on specific activations. While this helps reduce overfitting, it can also lower performance when the dataset is not large enough or when the model is already well-regularized (as in our case, after adding batch normalization). Therefore, dropout introduced a mild underfitting effect, which is consistent with its purpose as a regularization technique.

1.3 Question 3: Training with Validation and Data Augmentation

1.3.1 Training Plot with Validation Loss

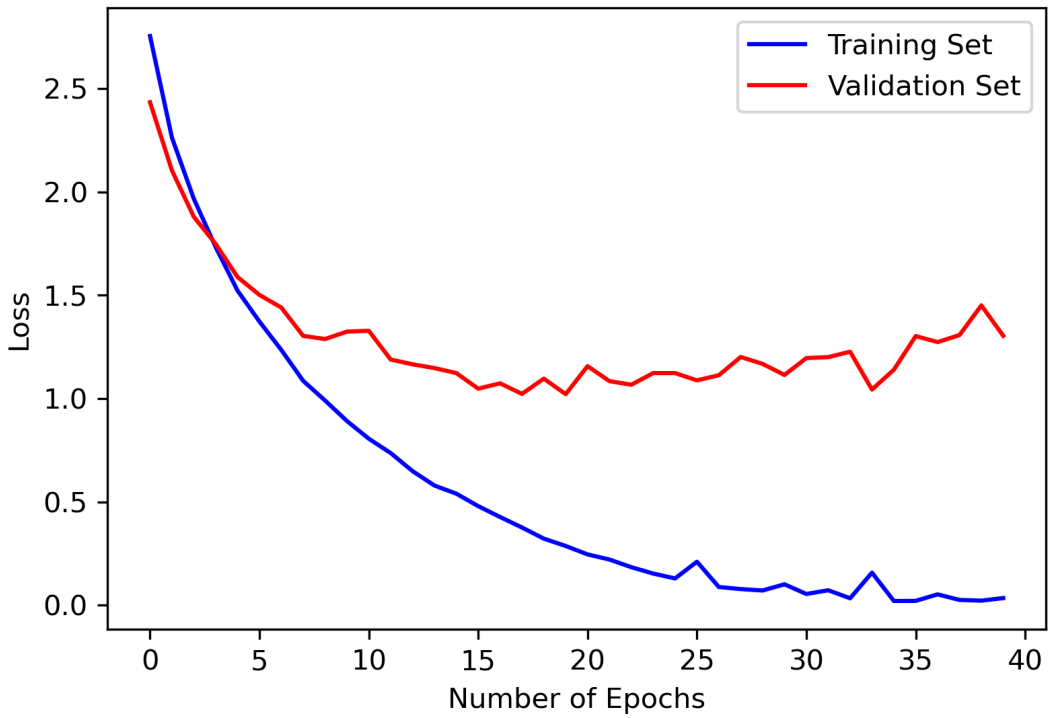

Figure 5: Training and Validation Loss Curve

The accuracy using the model with the lowest validation loss is 72%, which has a 3% decrease. There are several reasons that could lead to this result:

- **Random variation between validation and test splits:** Since the data splitting operation is random, each split may contain slightly different examples, where some may be easier and some may be more complicated.
- **Underfitting due to early stop:** Selecting the weights from the epoch with lowest validation loss could let the model has lower generalization capabilities. i.e., The model will prioritize the model fit to a specific validation data split.

1.3.2 Training Plot with Validation Loss After Data Augmentation

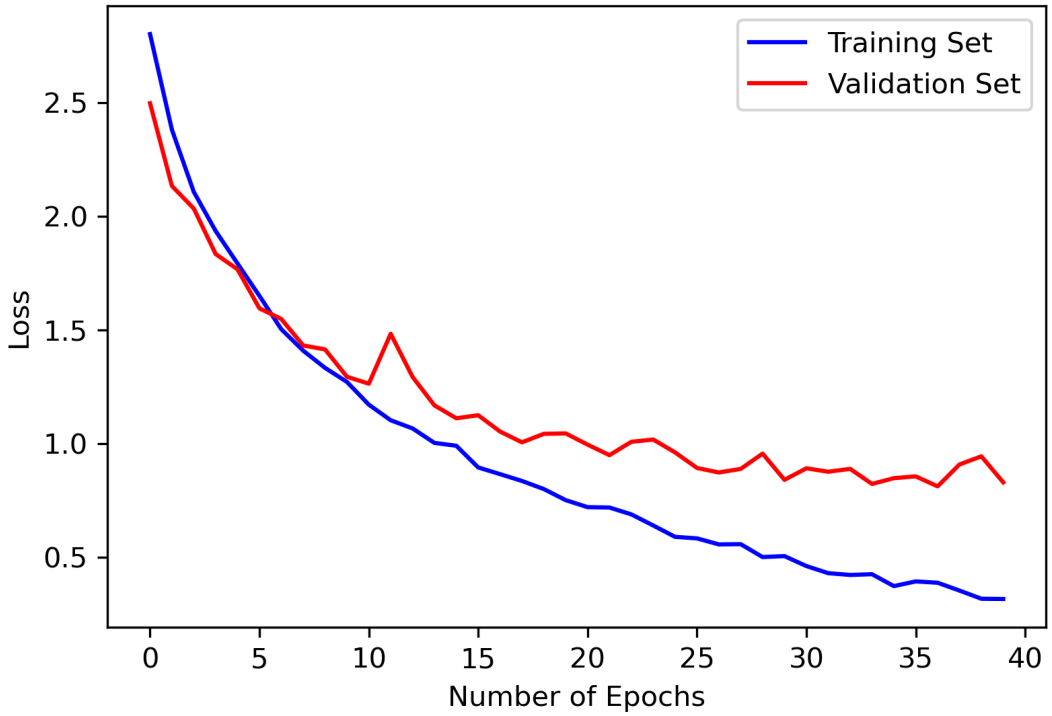

Figure 6: Training and Validation Loss Curve with Data Augmentation

I performed three data augmentation techniques, including random horizontal flip, rotation, and color jitter. The test accuracy after augmentation is at 80%, which has an 8% increase compared to the vanilla datasets. From the plot, we can see that performing augmentation reduced overfitting, as the validation loss isn't at a plateau; instead, it keeps decreasing during the 40 epochs. The reason why data augmentation works is because it introducess controllable noise into the training data. Hence, it increases the model's performance on generalization. However, performing excessive augmentation will lead to an accuracy drop, so the parameters of image transformations need to be tuned.

```
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.02),
    transforms.ToTensor(),
])
```

1.4 Question 4: Hyperparameter Tuning

1.4.1 Changing Optimizer

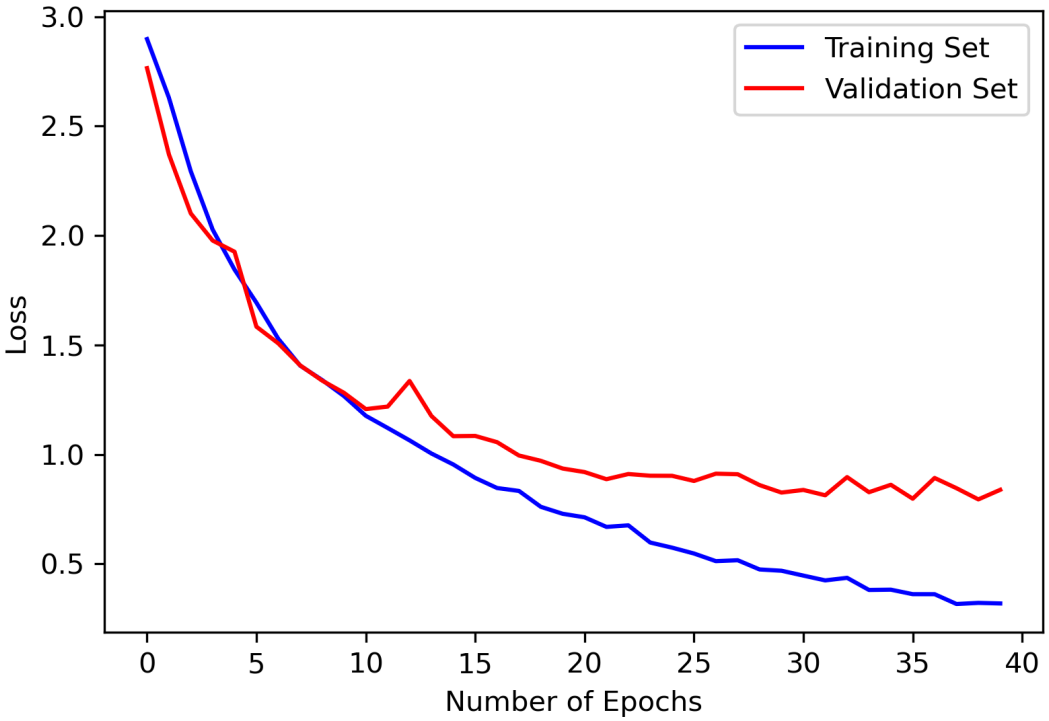

Figure 7: Training and Validation Loss Curve with AdamW Optimizer

The test accuracy after swapping the optimizer from RMSProp to AdamW is identical at 80%.

1.4.2 Tuning Learning Rate

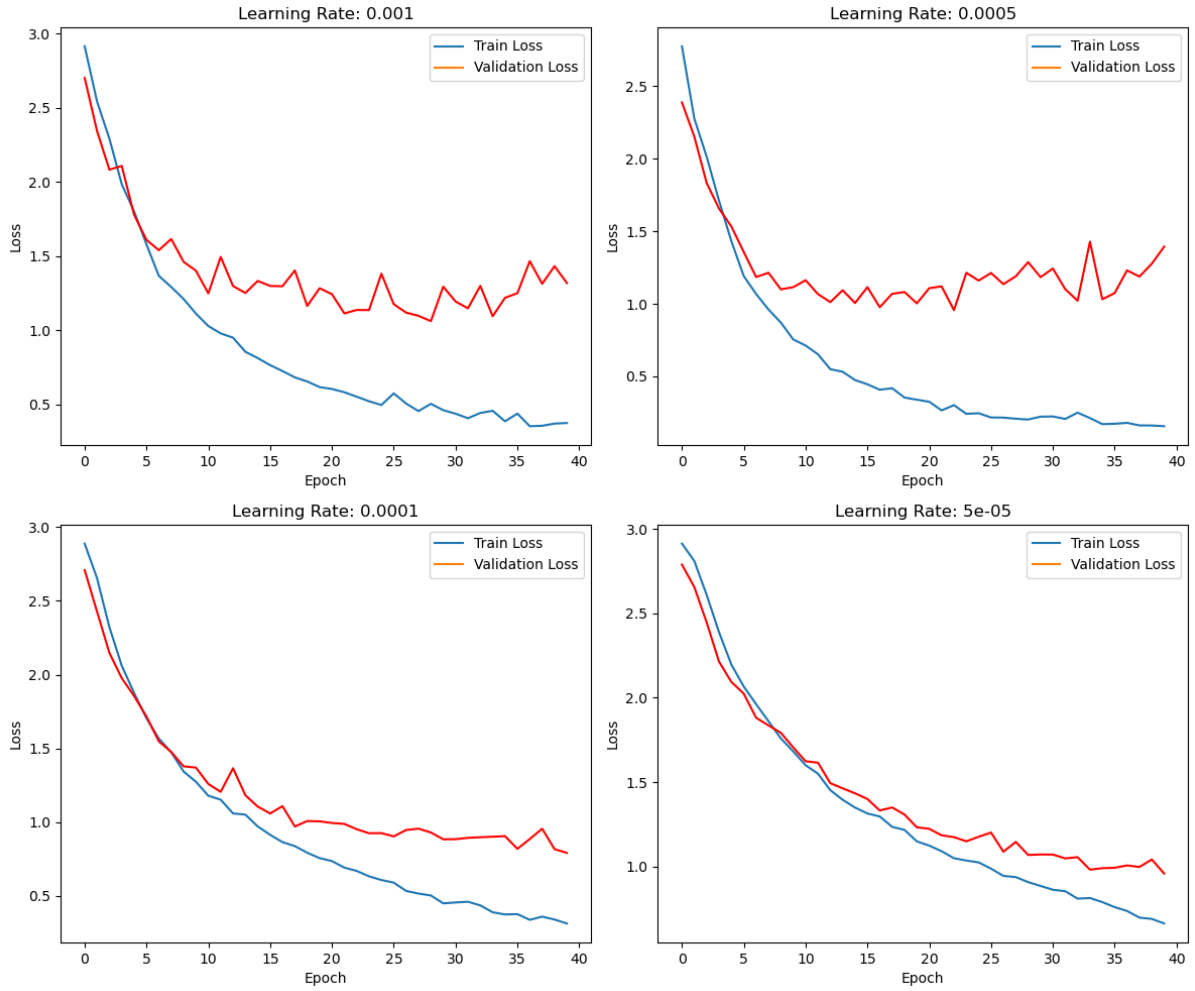

Figure 8: Training and Validation Loss Curve with Different Learning Rates

The test results for different learning rates are as follows:

- **lr = 0.001:** 73%
- **lr = 0.0001:** 78%
- **lr = 0.0005:** 78%
- **lr = 0.00005:** 65%

From the figure above, we can observe that as the learning rate decreases, the training curve becomes less volatile. This is because the learning rate determines the step size — how far the optimizer moves in the direction of the gradient at each iteration. When the learning rate is too high, the optimizer may overshoot the local or global minimum, causing oscillations or divergence in the loss curve. Meanwhile, a smaller learning rate will result in smoother convergence, but it'll require more epochs to get to the minimum.

1.4.3 Tuning Batch Size

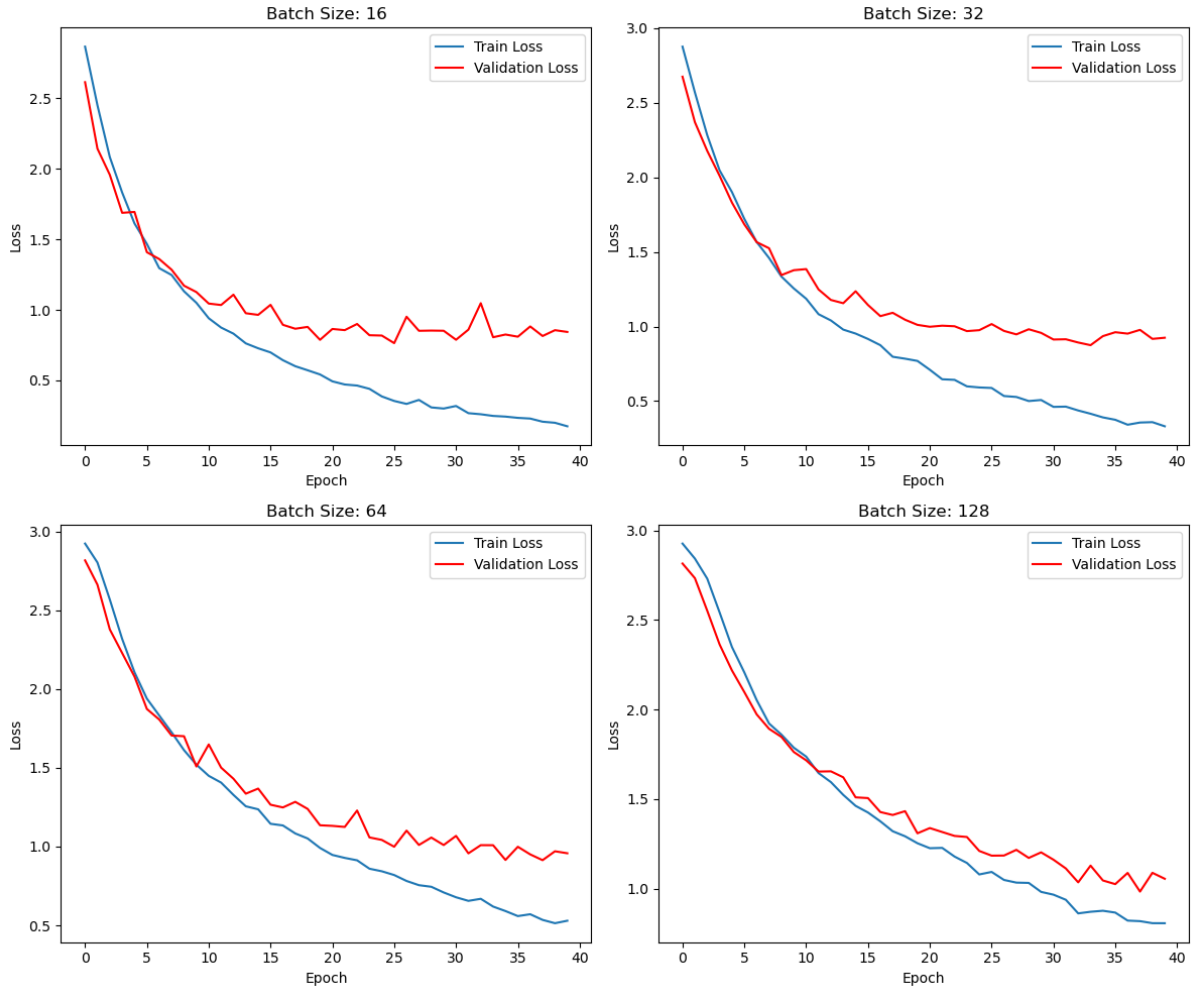

Figure 9: Training and Validation Loss Curve with Different Batch Sizes

From the figure above, we can see that as the batch size increases, both training and validation losses become smoother due to more stable gradient estimates. However, the test accuracy consistently decreases—from 75% (batch size 16) to 62% (batch size 128). This indicates that with a small batch size, the gradient updates are noisier, which acts as a form of implicit regularization and helps the optimizer escape sharp minima in the loss landscape. In contrast, larger batches provide more accurate gradient estimates but tend to converge to sharper minima, leading to poorer generalization. (Note: the test accuracy is tested with the parameters that have the lowest validation loss)

1.4.4 Hyperparameter Tuning

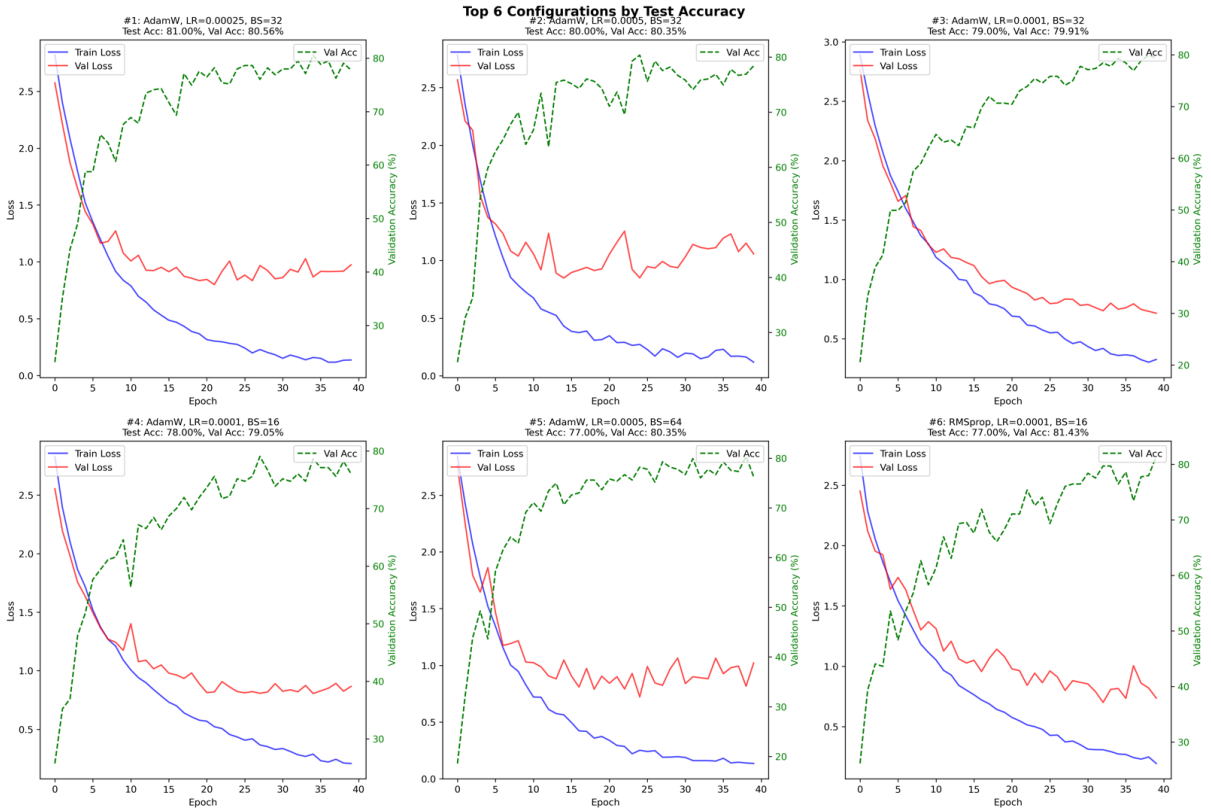

Figure 10: Hyperparameter Tuning

I implemented 27 combinations of hyperparameters to get the best test accuracy. The list of selected hyperparameters is as follows:

- Optimizer: AdamW, Adam, RMSProp
- Learning Rate: 0.0001, 0.00025, 0.0005
- Batch Size: 16, 32, 64

And the best performing hyperparameter combination achieved an 81% test accuracy by using the AdamW optimizer with no weight decay, 32 as the batch size, and 0.00025 as the learning rate.